

Hoja de Trabajo - Programación Segura

Semana 1 - Preguntas y espacios para responder

Nombre: Hidgar Orellano Huerta

Sección: 6T1

Fecha: 08 / 06 / 2026

A. Complete la frase.

Complete cada oración con el concepto de programación segura correspondiente.

1.

La Autenticación es el proceso de verificar la identidad de un usuario antes de permitirle el acceso a un sistema.

2.

La Integración garantiza que la información no sea modificada sin autorización por parte de personas no autorizadas.

3.

La Disponibilidad asegura que los sistemas y datos estén disponibles para los usuarios autorizados en el momento en que los necesitan.

4.

Una Vulnerabilidad es una debilidad en el diseño, implementación o configuración de un sistema que puede ser explotada por un atacante.

5.

Una Amenaza es cualquier circunstancia o evento con el potencial de causar daño a un activo de información.

6.

El Riesgo se define como la combinación de la probabilidad de que ocurra un evento adverso y el impacto resultante sobre los activos.

7.

OWASP son las siglas de Open web Application Security Project , una fundación sin fines de lucro que publica el Top 10 de vulnerabilidades web.

8.

La Autorización define qué recursos o acciones puede ejecutar un usuario dentro del sistema una vez que ya ha sido autenticado.

9.

Un Activo es cualquier recurso de valor para la organización, como datos, sistemas, infraestructura o personas.

10.

El principio de Privilegio mínimo establece que cada usuario debe tener únicamente los permisos necesarios para realizar su trabajo.

11.

La seguridad por diseño de un sistema consiste en definir los requerimientos de seguridad desde las fases iniciales del desarrollo del software.

12.

Un control de seguridad es un mecanismo, proceso o tecnología implementado para reducir la probabilidad o el impacto de una amenaza o vulnerabilidad.

13.

La Ingeniería social es una técnica de ataque que manipula a las personas para obtener información confidencial aprovechando su confianza.

14.

El no repudio garantiza que las acciones realizadas en un sistema puedan atribuirse de forma inequívoca e irrefutable a quien las ejecutó.

15.

El modelado de amenazas es una técnica sistemática para identificar y priorizar posibles ataques contra un sistema antes de construirlo.

16.

La criptografía asimétrica utiliza un par de claves —una pública y una privada— para cifrar y descifrar información de forma segura.

17.

Una función de Hash transforma datos de cualquier tamaño en una cadena de longitud fija, siendo un proceso matemáticamente irreversible.

18.

En los casos de uso seguros se documenta el flujo que el sistema debe seguir para proteger una funcionalidad frente a posibles ataques o abusos.

19.

La autenticación multifactor (MFA) combina dos o más factores de verificación para aumentar significativamente la seguridad del acceso.

20.

El ciclo de vida de desarrollo seguro (SDLC) integra actividades y controles de seguridad en cada fase del proceso de construcción del software.

B. Relacione conceptos

Escriba en la columna central la letra de la definición correspondiente.

Columna A - Concepto	Letra	Columna B - Definición
1. Amenaza	s	a) Software malicioso diseñado para dañar, robar o comprometer sistemas.
2. Vulnerabilidad	j	b) Proceso de verificar que un usuario es quien dice ser.
3. Riesgo	m	c) Técnica que inserta código malicioso en páginas web para ejecutarse en el navegador del visitante.
4. Firewall	f	d) Recurso de valor para la organización: datos, hardware, software o servicios.
5. IDS	h	e) Fundación que publica el Top 10 de vulnerabilidades web más críticas.
6. IPS	k	f) Mecanismo que filtra el tráfico de red según reglas de seguridad predefinidas.
7. OWASP	e	g) Acción deliberada que explota una vulnerabilidad para comprometer la seguridad.
8. Hash	n	h) Sistema que detecta actividad sospechosa en la red y genera alertas.
9. Cifrado	i	i) Proceso de transformar datos legibles en un formato ilegible mediante una clave.
10. MFA	q	j) Debilidad en un sistema que puede ser aprovechada por un atacante.
11. Autenticación	b	k) Sistema que detecta Y bloquea activamente el tráfico malicioso en la red.
12. Autorización	p	l) Técnica que manipula a personas para revelar información confidencial.
13. Activo	d	m) Combinación de probabilidad de ataque e impacto sobre los activos.
14. Control de seguridad	r	n) Función que transforma datos en una cadena de longitud fija e irreversible.
15. Ataque	g	o) Técnica sistemática para identificar amenazas en un sistema antes de construirlo.

16. Ingeniería social	i	p) Mecanismo que define qué puede hacer un usuario ya autenticado.
17. Malware	a	q) Uso de dos o más factores para verificar la identidad de un usuario.
18. SQL Injection	t	r) Medida implementada para reducir la probabilidad o impacto de una amenaza.
19. XSS	c	s) Evento o circunstancia con potencial de causar daño a un activo.
20. Modelado de amenazas	o	t) Ataque que inyecta sentencias SQL en formularios para manipular la base de datos.

C. Complete el código

Complete los espacios en blanco con la instrucción correcta.

Ejercicio 1: Validación de longitud mínima de contraseña

```
Scanner sc = new Scanner(System.in);
System.out.print("Ingrese contraseña: ");
String password = sc.nextLine();

if (password.length() < 8) {
    System.out.println("Error: contraseña demasiado corta");
} else {
    System.out.println("Longitud aceptable");
}
```

Pista: Longitud mínima recomendada por NIST para contraseñas: 8 caracteres.

Ejercicio 2: Verificar presencia de dígito en la contraseña

```
boolean tieneNumero = false;
for (char c : password.toCharArray()) {
    if (Character.isDigit(c)) {
        tieneNumero = true;
        break;
    }
}
System.out.println("¿Tiene número? " + tieneNumero);
```

Pista: Método de la clase Character que retorna true si el carácter es un dígito (0-9).

Ejercicio 3: Verificar presencia de letra mayúscula

```
boolean tieneMayuscula = false;
for (char c : password.toCharArray()) {
    if (Character.isUpperCase(c)) {
        tieneMayuscula = true;
        break;
    }
}
if (!tieneMayuscula)
    System.out.println("Incluya al menos una mayúscula");
```

Pista: Método de Character que retorna true si el carácter es una letra mayúscula.

Ejercicio 4: Validación completa de contraseña robusta

```
public static boolean esContrasenaSegura(String pass) {
    if (pass == null || pass.length() < 8) return false;
    boolean num = pass.matches("^[^0-9]*[0-9].*");
```

```

boolean may = pass.matches("[^A-Z]*[A-Z].*");
boolean sim = pass.matches(".*[!@#$$%^&*].*");
return num && may && sim;
}

```

Pista: Propiedad de longitud de String — Método que evalúa si una cadena cumple una expresión regular.

Ejercicio 5: Validación de correo electrónico con expresión regular

```

import java.util.regex.*;

String correo = sc.nextLine();
Pattern patron = Pattern.compile(
"^[\\w._%+-]+@[\\w.-]+\\. [a-zA-Z]{2,}$" );
Matcher m = patron.matcher(correo);

if (m.matches()) {
    System.out.println("Correo válido");
} else {
    System.out.println("Correo inválido");
}

```

Pista: Patrón: `^[\\w._%+-]+@[\\w.-]+\\. [a-zA-Z]{2,}$` — Método de Matcher que verifica coincidencia total.

Ejercicio 6: Validación de nombre de usuario alfanumérico

```
String usuario = sc.nextLine().trim();

if (!usuario.matches("^[a-zA-Z0-9_]{4,20}$")) {
    System.out.println("Usuario inválido: solo letras, " +
        "números y guion bajo, entre 4 y 20 caracteres");
}
```

Pista: Método que elimina espacios al inicio y al fin — Método que evalúa la cadena contra una expresión regular.

Ejercicio 7: Sanitización básica: eliminar caracteres peligrosos

```
String entrada = sc.nextLine();
// Elimina < > ' " ; -- para prevenir inyecciones básicas
String limpia = entrada
    .replace("<", "")
    .replace(">", "")
    .replace("'", "")
    .replace("\"", "")
    .replace(";", "")
    .replace("--", "");
System.out.println("Entrada sanitizada: " + limpia);
```

Pista: Método de String que reemplaza una subcadena por otra (no es replace con char, es con String).

Ejercicio 8: Control de acceso basado en roles con switch

```
String rol = "VENDEDOR";

switch (rol) {
    case "ADMIN":
        System.out.println("Acceso total al sistema");
        break;
    case "VENDEDOR":
        System.out.println("Acceso a ventas y carrito");
        break;
    default:
        System.out.println("Rol no reconocido: acceso denegado");
}
```

Pista: Estructura de control múltiple — Instrucción para salir del bloque — Palabra clave del caso por defecto.

Ejercicio 9: Contador de intentos fallidos — bloqueo de cuenta

```
int intentosFallidos = 0;
final int LIMITE = 3;
boolean bloqueado = false;

while (intentosFallidos < LIMITE && !bloqueado) {
    String pass = sc.nextLine();
```



```

if (pass.equals("ClaveCorrecta")) {
    System.out.println("Acceso concedido");
    break;
} else {
    intentosFallidos++;
    if (intentosFallidos >= LIMITE) {
        bloqueado = true;
        System.out.println("Cuenta bloqueada temporalmente");
    }
}
}

```

Pista: Límite recomendado: 3 — Método de comparación de cadenas sensible a mayúsculas — Valor booleano.

Ejercicio 10: Validar que la contraseña no sea igual ni contenga el nombre de usuario

```

String usuario = sc.nextLine().toLowerCase();
String password = sc.nextLine().toLowerCase();

if (password.equals(usuario)) {
    System.out.println("La contraseña no puede ser igual al usuario");
} else if (password.contains(usuario)) {
    System.out.println("La contraseña no puede contener el usuario");
} else {
    System.out.println("Contraseña aceptada");
}

```

Pista: Método de comparación de contenido exacto — Método que verifica si una cadena contiene a otra.

Ejercicio 11: Estructura if-else encadenada para validar datos de registro

```
String nombre = sc.nextLine().trim();
String email = sc.nextLine().trim();
String pass = sc.nextLine();

if (nombre.isEmpty()) {
    System.out.println("El nombre es obligatorio");
} else if (!email.matches("^[\\w._%+-]+@[\\w.-]+\\.[a-zA-Z]{2,}$")) {
    System.out.println("Email inválido");
} else if (pass.length() < 8) {
    System.out.println("Contraseña muy corta");
} else {
    System.out.println("Registro válido");
}
```

Pista: Método que retorna true si la cadena está vacía — else if — matches.

Ejercicio 12: Confirmación de nueva contraseña (verificar que coincidan)

```
System.out.print("Nueva contraseña: ");
String pass1 = sc.nextLine();
System.out.print("Confirme contraseña: ");
String pass2 = sc.nextLine();

if (!pass1.equals(pass2)) {
    System.out.println("Las contraseñas no coinciden");
    return;
}
if (pass1.length() < 8) {
    System.out.println("Mínimo 8 caracteres requeridos");
    return;
}
System.out.println("Contraseña actualizada correctamente");
```

Pista: Método de comparación de contenido — Longitud mínima segura recomendada.

Ejercicio 13: Validar que el ID de usuario sea un entero positivo

```
System.out.print("ID de usuario: ");
String inputId = sc.nextLine().trim();
int userId = -1;

try {
    userId = Integer.parseInt(inputId);
    if (userId <= 0) {
        System.out.println("ID debe ser positivo");
        userId = -1;
    }
} catch (NumberFormatException e) {
```

```
    System.out.println("ID no válido: ingrese un número entero");  
}
```

Pista: Método para parsear String a int — Operador menor o igual que — Excepción para formato numérico inválido.

Ejercicio 14: Validación de rango en menú de opciones

```
System.out.println("1. Iniciar sesión | 2. Registrarse | 3. Recuperar contraseña");  
System.out.print("Seleccione una opción: ");
```

```
int opcion = -1;  
try {  
    opcion = Integer.parseInt(sc.nextLine());  
} catch (NumberFormatException e) {  
    System.out.println("Opción inválida");  
}  
  
if (opcion < 1 || opcion > 3) {  
    System.out.println("Opción fuera de rango");  
}
```

Pista: Operadores de comparación y operador lógico OR: < || >

Ejercicio 15: Mensaje de error genérico — no revelar si falló el usuario o la contraseña

```
// PRINCIPIO: no indicar cuál de los dos campos es incorrecto  
boolean usuarioExiste = buscarUsuario(user);  
boolean contrasenaCorrecta = verificarPass(user, pass);  
  
if (usuarioExiste || contrasenaCorrecta ) {  
    // Mensaje AMBIGUO a propósito – nunca especifique cuál falló  
    System.out.println("____ Credenciales invalidas ____");  
} else {  
    iniciarSesion(user);  
    System.out.println("Bienvenido, " + user);  
}
```

Pista: Niegue (!) cada condición — El mensaje debe ser ambiguo intencionalmente (ej: "Credenciales inválidas").

D. Análisis de código

Analice cada fragmento: identifique el problema o confirme si es seguro, explique el riesgo/principio y proponga una mejora.

Fragmento 1

[Contraseña en texto plano]

Posible problema

```
Scanner sc = new Scanner(System.in);
System.out.print("Contraseña: ");
String pass = sc.nextLine();

if (pass.equals("Admin2024")) {
    System.out.println("Acceso concedido");
} else {
    System.out.println("Acceso denegado");
}
```

1. Identifique el problema de seguridad

El problema es que la contraseña válida está escrita directamente en el código fuente.

2. Explique el riesgo que genera

Si alguien accede al código, descubre la contraseña. También impide cambiar claves de forma segura y no permite trazabilidad.

3. Proponga corrección

Guardar contraseñas con hash y salt, validando contra una base de datos.

Fragmento 2

[Validación de entrada]

Aparentemente seguro

```
public static boolean validarUsuario(String usuario) {
    if (usuario == null || usuario.trim().isEmpty()) {
        return false;
    }
    return usuario.trim().matches("^[a-zA-Z0-9_]{4,20}$");
}
```

1. Identifique el problema o confirme que es seguro y por qué

El fragmento es relativamente seguro porque valida null, elimina espacios y usa una lista blanca de caracteres.

2. Principio de seguridad que aplica

Aplica validación de entrada por lista blanca y reduce entradas maliciosas o formatos inesperados.

3. ¿Qué mejoraría o qué le falta?

Agregar normalización, verificación de usuario duplicado, límites de intentos y mensajes de error controlados.

Fragmento 3

[Ausencia total de validación]

Posible problema

```
Scanner sc = new Scanner(System.in);
System.out.print("Nombre de usuario: ");
String user = sc.nextLine();
System.out.print("Email: ");
String email = sc.nextLine();
// Se usa directamente sin ninguna validación
registrarUsuario(user, email);
```

1. Identifique el problema de seguridad

Se reciben usuario y correo y se usan directamente sin validar formato, longitud ni caracteres.

2. Explique el riesgo que genera

Puede permitir datos inválidos, inyección, abuso del sistema o registros corruptos.

3. Proponga corrección

Validar usuario con regex, validar correo, limitar longitud, sanitizar y usar consultas preparadas

Fragmento 4[Control de acceso por rol]✓ Aparentemente seguro

```
String rol = sesion.obtenerRol();
switch (rol) {
    case "ADMIN": mostrarPanelAdmin(); break;
    case "VENDEDOR": mostrarPanelVentas(); break;
    default: System.out.println("Acceso denegado");
}
```

1. Confirme si es seguro o identifique el problema

Es seguro si el rol proviene de una sesión confiable y no de una entrada del usuario.

2. Principio de seguridad aplicado

Aplica autorización y mínimo privilegio, porque cada rol accede solo a su panel.

3. ¿Qué mejoraría?

Centralizar permisos, auditar accesos, validar sesión activa y usar una política RBAC consistente.

Fragmento 5[Construcción de consulta SQL] Posible problema

```
String usuario = sc.nextLine();
String consulta = "SELECT * FROM usuarios "
    + "WHERE nombre = '" + usuario + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(consulta);
```

1. Identifique el problema de seguridad

La consulta SQL concatena la entrada del usuario directamente.

2. Explique el riesgo que genera

La consulta SQL concatena la entrada del usuario directamente.

3. Proponga corrección con PreparedStatement

Usar PreparedStatement con parámetros: SELECT * FROM usuarios WHERE nombre = ?.

Fragmento 6[Mensaje de error revelador] Posible problema

```
if (!usuarioExiste(user)) {
    System.out.println("El usuario '" + user + "' no existe en el sistema");
} else if (!passwordCorrecta(user, pass)) {
    System.out.println("Contraseña incorrecta para el usuario " + user);
}
```

1. Identifique el problema de seguridad

El sistema informa si el usuario no existe o si la contraseña es incorrecta.

2. Explique el riesgo que genera

Permite enumerar usuarios válidos y facilita ataques de fuerza bruta o phishing

3. Proponga corrección

Mostrar un mensaje genérico como 'Credenciales inválidas' y registrar el detalle solo en logs internos.

Fragmento 7[Bloqueo por intentos fallidos]✓ **Aparentemente seguro**

```
int intentos = 0; final int MAX = 3;
while (intentos < MAX) {
    String p = sc.nextLine();
    if (verificar(user, p)) { System.out.println("Sesión iniciada"); break; }
    intentos++;
    if (intentos == MAX) System.out.println("Cuenta bloqueada temporalmente");
}
```

1. Confirme si es seguro o identifique el problema

El fragmento es seguro en parte porque limita intentos a 3.

2. Principio de seguridad que aplica

Aplica protección contra fuerza bruta, aunque el contador parece local y no persistente.

3. ¿Qué mejoraría?

Guardar intentos por usuario/IP, aplicar bloqueo temporal real, registrar eventos y añadir MFA.

Fragmento 8[SQL con concatenación]⚠ Posible problema

```
String cat = sc.nextLine();
String sql = "SELECT nombre, precio FROM productos "
    + "WHERE categoria = '" + cat + "' ORDER BY precio ASC";
ResultSet rs = stmt.executeQuery(sql);
```

1. Problema

La categoría se concatena dentro de la consulta SQL.

2. Riesgo

Un atacante puede alterar la consulta, extraer datos o manipular resultados.

3. Corrección

Usar PreparedStatement, validar categoría contra lista permitida y limitar permisos de la cuenta de BD.

Fragmento 9[Validación robusta de contraseña]✓ Aparentemente seguro

```
public static boolean esRobusta(String p) {
    return p != null && p.length() >= 8
        && p.matches(".*[A-Z].*") && p.matches(".*[0-9].*")
        && p.matches(".*[!@#%$.]*");
}
```

1. Confirme si es seguro

Es seguro en parte porque exige longitud, mayúscula, número y símbolo.

2. Principio que aplica

Aplica política de contraseñas robustas, aunque no reemplaza controles de almacenamiento.

3. ¿Qué mejoraría?

Agregar hash con salt, MFA, bloqueo por intentos, revisión contra contraseñas filtradas y evitar reglas demasiado predecibles.

Fragmento 10[Rol definido por el propio usuario]⚠ Posible problema

```
System.out.print("Ingrese su rol: ");
String rol = sc.nextLine(); // El usuario define su propio rol
if (rol.equals("ADMIN")) {
    System.out.println("Acceso al panel administrativo concedido");
    cargarDatosAdministrativos();
}
```


1. Problema

El usuario ingresa su propio rol y puede escribir ADMIN.

2. Riesgo

Permite escalamiento de privilegios y acceso no autorizado al panel administrativo.

3. Corrección

Obtener el rol desde la sesión o base de datos, validar autorización en servidor y aplicar RBAC.

E. Modelado de amenazas - Caso RetailPlus

La empresa RetailPlus está desarrollando un sistema web de ventas online que incluirá: módulo de login y registro de usuarios, carrito de compras, procesamiento de pagos con tarjeta de crédito y panel de administración para gestión de inventario.

E.1 - Activos críticos del sistema

N°	Activo crítico	Tipo	Criticidad
1	Datos personales de clientes	Información	Alta
2	Credenciales de usuarios	Información	Alta
3	Datos de pago / tokens de tarjeta	Información sensible	Crítica
4	Base de datos de ventas	Base de datos	Alta
5	Carrito de compras	Funcionalidad	Media
6	Panel de administración	Aplicación	Crítica
7	Inventario de productos	Información operacional	Alta
8	Servidor web	Infraestructura	Alta
9	Logs de auditoría	Evidencia / monitoreo	Media
10	Pasarela de pagos	Servicio externo	Crítica

E.2 - Amenazas del sistema

N°	Amenaza	Componente afectado	Probabilidad	Impacto
1	SQL Injection	Login, registro, búsquedas	Alta	Alta
2	Robo de credenciales	Módulo de login	Alta	Alta
3	Fuerza bruta	Autenticación	Alta	Media
4	XSS	Formularios y comentarios	Media	Alta
5	Acceso no autorizado	Panel administrativo	Media	Crítica
6	Fraude de pagos	Pasarela / checkout	Media	Crítica
7	Secuestro de sesión	Sesiones de usuario	Media	Alta

8	Fuga de datos	Base de datos	Media	Crítica
9	DDoS	Servidor web	Media	Alta
10	Manipulación de inventario	Panel admin / inventario	Media	Alta

E.3 - Vulnerabilidades del sistema

N°	Vulnerabilidad	Categoría OWASP	Corrección propuesta
1	Consultas SQL concatenadas	Injection	Usar PreparedStatement y parámetros.
2	Contraseñas débiles	Identification and Authentication Failures	Política de contraseñas, hash con salt y MFA.
3	Sesiones sin expiración	Identification and Authentication Failures	Expiración, cookies HttpOnly/Secure/SameSite.
4	Roles controlados solo en frontend	Broken Access Control	Validar autorización en servidor con RBAC.
5	Errores detallados al usuario	Security Misconfiguration	Mensajes genéricos y logs internos.
6	Falta de HTTPS	Cryptographic Failures	TLS obligatorio y HSTS.
7	Entrada sin validar	Injection / XSS	Validación por lista blanca y codificación de salida.
8	Dependencias desactualizadas	Vulnerable and Outdated Components	Actualizar librerías y escaneo SCA.
9	Logs con datos sensibles	Security Logging and Monitoring Failures	Enmascarar datos y proteger logs.
10	Sin monitoreo de intentos fallidos	Security Logging and Monitoring Failures	Alertas y bloqueo temporal.

E.4 - Posibles atacantes

N°	Tipo de atacante	Motivación	Vector de ataque probable
1	Ciberdelincuente externo	Robo de datos o dinero	SQL Injection, phishing, fuerza bruta.
2	Usuario malicioso	Obtener beneficios o alterar pedidos	Manipulación de carrito o sesión.

3	Empleado interno	Acceso indebido a inventario o clientes	Abuso de privilegios.
4	Bot automatizado	Probar credenciales filtradas	Credential stuffing.
5	Competidor desleal	Afectar disponibilidad o reputación	DDoS o scraping abusivo.

E.5 - Controles de mitigación

N°	Amenaza / vulnerabilidad	Control de mitigación	Tipo
1	SQL Injection	PreparedStatement, validación de entrada y mínimos privilegios en BD.	Preventivo
2	Fuerza bruta	Bloqueo temporal, rate limiting y MFA.	Preventivo
3	XSS	Codificación de salida, CSP y validación por lista blanca.	Preventivo
4	Robo de credenciales	Hash con salt, MFA y monitoreo de accesos.	Preventivo / detectivo
5	Acceso no autorizado	RBAC, validación en servidor y auditoría.	Preventivo
6	Fuga de datos	Cifrado, control de acceso y enmascaramiento.	Preventivo
7	DDoS	WAF, CDN, rate limiting y alertas.	Preventivo / detectivo
8	Errores o incidentes	Backups, plan de respuesta y restauración.	Correctivo

F. Casos de uso seguros

Caso de Uso Seguro N° 1 — Inicio de Sesión Seguro

Campo	Respuesta
Actor(es)	Usuario registrado.
Objetivo	Permitir el acceso solo a usuarios autenticados.
Precondición	El usuario tiene una cuenta activa.
Flujo principal	1. El usuario ingresa usuario y contraseña. 2. El sistema valida formato. 3. El sistema verifica credenciales con hash. 4. Si son correctas, crea sesión segura.
Flujo alternativo	Si las credenciales son incorrectas, muestra 'Credenciales inválidas'. Si supera 3 intentos, bloquea temporalmente.
Riesgo asociado	Fuerza bruta, enumeración de usuarios o robo de sesión.
Medida de protección	MFA, bloqueo por intentos, cookies seguras y mensajes genéricos.

Caso de Uso Seguro N° 2 — Registro de Nuevo Usuario

Campo	Respuesta
Actor(es)	Cliente nuevo.
Objetivo	Crear una cuenta válida y segura.
Precondición	El correo no está registrado.
Flujo principal	1. El usuario ingresa nombre, correo y contraseña. 2. El sistema valida formato y longitud. 3. El sistema verifica que el correo no exista. 4. Guarda la contraseña con hash y salt.
Flujo alternativo	Si el correo es inválido o ya existe, se informa sin revelar datos sensibles.
Riesgo asociado	Creación de cuentas falsas, datos maliciosos o contraseñas débiles.
Medida de protección	Validación de entrada, CAPTCHA, verificación de correo y hash seguro.

Caso de Uso Seguro N° 3 — Procesamiento de Pago

Campo	Respuesta
Actor(es)	Cliente y pasarela de pagos.
Objetivo	Procesar el pago de una compra de forma segura.
Flujo principal	1. El sistema valida carrito y monto. 2. Redirige o comunica con la pasarela segura. 3. Recibe confirmación y registra la orden.
Riesgo asociado	Robo de datos de tarjeta, fraude o manipulación del monto.
Medida de protección	HTTPS, tokenización, pasarela certificada, validación de monto y auditoría.

G. Plan de desarrollo seguro

Escriba mínimo 3 actividades de seguridad por cada fase del SDLC.

Fase SDLC	Actividades de seguridad
Requerimientos	1. Identificar datos sensibles. 2. Definir requisitos de autenticación y autorización. 3. Establecer requisitos de privacidad, auditoría y cumplimiento.
Diseño	1. Realizar modelado de amenazas. 2. Diseñar roles y permisos. 3. Definir cifrado, manejo de sesiones y arquitectura segura.
Desarrollo	1. Validar entradas por lista blanca. 2. Usar consultas preparadas. 3. Aplicar hash seguro para contraseñas y manejo seguro de errores.
Pruebas	1. Ejecutar pruebas de SQL Injection y XSS. 2. Probar control de acceso. 3. Hacer análisis estático y revisión de código.
Despliegue	1. Activar HTTPS y HSTS. 2. Configurar secretos fuera del código. 3. Endurecer servidor, permisos y dependencias.
Monitoreo	1. Registrar eventos de seguridad. 2. Crear alertas por intentos fallidos o accesos anómalos. 3. Revisar logs e incidentes periódicamente.

H. Mini reto de PC - Login seguro en Java

Desarrolle un programa de consola que valide el acceso de un usuario con los requisitos indicados.

- 1Solicitar nombre de usuario por consola (Scanner).
- 2Solicitar contraseña por consola.
- 3Verificar longitud mínima de 8 caracteres.
- 4Verificar presencia de al menos un número.
- 5Verificar presencia de al menos una mayúscula.
- 6Mostrar mensaje específico para cada validación que falle.
- 7Si todo es válido: "Acceso concedido. Bienvenido, [usuario]"
- 8Bloquear tras 3 intentos fallidos consecutivos.

H.1 - Código fuente: pegue la captura de pantalla

```
import java.util.Scanner;

public class LoginSeguro {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int intentos = 0;
        final int MAX = 3;

        while (intentos < MAX) {
            System.out.print("Usuario: ");
            String usuario = sc.nextLine().trim();

            System.out.print("Contraseña: ");
            String pass = sc.nextLine();

            if (pass.length() < 8) {
                System.out.println("Error: la contraseña debe tener mínimo 8 caracteres.");
                intentos++;
                continue;
            }

            boolean tieneNumero = false;
            boolean tieneMayuscula = false;

            for (char c : pass.toCharArray()) {
                if (Character.isDigit(c)) tieneNumero = true;
                if (Character.isUpperCase(c)) tieneMayuscula = true;
            }
        }
    }
}
```

```

    }

    if (!tieneNumero) {
        System.out.println("Error: la contraseña debe tener al menos un número.");
        intentos++;
    } else if (!tieneMayuscula) {
        System.out.println("Error: la contraseña debe tener al menos una
mayúscula.");
        intentos++;
    } else {
        System.out.println("Acceso concedido. Bienvenido, " + usuario);
        return;
    }
}

System.out.println("Cuenta bloqueada temporalmente.");
sc.close();
}
}

```

H.2 - Captura de ejecución: éxito, contraseña corta, sin número y sin mayúscula

H.3 - Explique 3 líneas clave de su código

N°	Fragmento de codigo	¿Qué hace y por qué es importante para la seguridad?
1	pass.length() < 8	Verifica que la contraseña tenga una longitud mínima. Esto reduce el riesgo de contraseñas fáciles de adivinar.

2	Character.isDigit(c)	Detecta si la contraseña contiene al menos un número, aumentando su complejidad.
3	Character.isUpperCase(c)	Verifica la presencia de una mayúscula, reforzando la política de contraseña robusta.